

ハッカソン 1

Processing による音信号作成

有本泰子

1 音の生成と波形描画

音信号を生成する方法をソースコード 1 に示す。(基本) 周波数が 440Hz で時間長を 5 秒とした正弦波の音信号を生成している。本プログラムでは、音信号を生成したのち、p キーを押下すると音が再生され、その波形が描画される。音声設定のデフォルト値は以下の通り。

- ステレオ
- バッファサイズ：1024 点
- サンプリング周波数：44100 Hz
- 量子化ビット数：16 bit

ソースコード 1 音の生成と波形の描画

```
1  import ddf.minim.*;
2  import ddf.minim.ugens.*;
3
4  Minim minim;
5  AudioOutput out;
6
7  void setup()
8  {
9      size(512, 200);
10     // minimのインスタンスを用意
11     minim = new Minim(this);
12     // minimのgetLineOutメソッドを呼び出し、AudioOutputオブジェクトを受け取る
13     out = minim.getLineOut();
14     // テンポの設定, BPM=120
15     out.setTempo( 120 );
16 }
17
18 void playSong() {
19     // 再生を停止
20     out.pauseNotes();
21     // 音を追加 (開始時刻, 音の長さ, 音の高さ) ----- (a), (b)
22     out.playNote(0.0f, 5.0, "A4");
23     // 再生
24     out.resumeNotes();
25 }
26
27 void draw()
28 {
29     background(0);
30     stroke(255);
31
32     // 左チャンネルと右チャンネルに入っている波形を描画 ----- (c)
33     for (int i = 0; i < out.bufferSize() - 1; i++)
34     {
35         line( i, 50 - out.left.get(i)*50, i+1, 50 - out.left.get(i+1)*50 );
36         line( i, 150 - out.right.get(i)*50, i+1, 150 - out.right.get(i+1)*50 );
```

```

37     }
38 }
39
40 void keyPressed() {
41     switch (key)
42     {
43     case 'p':
44         // 作成した信号を出力
45         playSong();
46         break;
47     }
48 }

```

(a) 音の再生タイミングと長さを決める

音の再生タイミングを変更するには、playNote メソッドの第 1 引数の値を変更すればよい。また、音の長さを変更するには、playNote メソッドの第 2 引数の値を変更する。例えば、音の再生開始を 1 秒遅らせ、長さを 2 秒に変更する場合は次に示すプログラムのように書けば良い。

ソースコード 2 音の再生タイミングと長さを決める

```

20 // 音を追加（開始時刻，音の長さ，音の高さ）
21 out.playNote(1.0f, 2.0, "A4");

```

(b) 音の高さを決める

音の高さを変更するには、周波数を変更すれば良いので、playNote メソッドの第 3 引数の値を変更すればよい。例えば、音の高さを C4 に変更する場合は次に示すプログラムのように書けば良い。

ソースコード 3 音の高さを決める

```

20 // 音を追加（開始時刻，音の長さ，音の高さ）
21 out.playNote(0.0f, 5.0, "C4");

```

(c) 波形の描画

出力デバイスに出力される音信号の波形を描画する。AudioOutput クラスの変数 out を使用して、出力バッファに入っている信号をバッファサイズ分だけ描画する。出力バッファのサイズを取得するには bufferSize() メソッドを使用する。line 関数を使って音信号を描画する。描画ウィンドウの高さを 200 に設定している（9 行目）ので、左チャンネルの振幅 0 がウィンドウの 50 の位置に、右チャンネルの振幅 0 がウィンドウの 150 の位置に来るように高さを調整している。変数 wave の左チャンネルの信号を取得するには、out.left.get(インデックス) と記述する。右チャンネルの信号を取得する場合には、out.right.get(インデックス) と記述する。

2 振幅と音色を変更する

各音の振幅と音色を変更する方法について説明する。このセクションに記載したソースコードを利用したプログラムの全文は、付録 A のソースコード 9 に記載した。

2.1 音色の情報を持つ音のクラスを作成する

音色を変更するため Instrument インタフェースを実装する。ソースコード 4 に Instrument インタフェースを実装した例を示す。まず、音色の情報を格納するための Waveform クラスの変数 currentWaveform を用意する（6 行目）。Instrument クラスは HackInstrument というクラス名で実装している（9 行目）。クラス内のフィールド（変数）として、Oscil クラスの変数 wave、Line クラスの変数 ampEnv、振幅の最大値を格納する変数 maxAmp を用意した（11-13 行目）。コンストラクタ内では引数として受け取った基本周波数（frequency）、最大振幅（maxAmp）、音色（wf）の

情報を使い、音信号を作成する。基本周波数と音色の情報を使って `Oscil` クラスのインスタンスを作成する（17 行目）。第 2 引数には振幅の値が求められるが、この後に振幅変調をかけるため引数として受け取った最大振幅の値は `Oscil` クラスのインスタンスの引数として与えない。引数として受け取った最大振幅の値は、クラスのフィールド（変数）である `maxAmp` に格納しておく（19 行目）。線型的な振幅変調を与える `Line` クラスのインスタンスを作成（21 行目）し、作成した音信号を `Line` クラスのインスタンスの出力に送る（23 行目）。

27 行目と 35 行目にある二つのメソッドはコールバック関数であり、`noteOn` メソッドは音を再生するときにシステムから呼ばれ、`noteOff` メソッドは再生を停止するときにシステムから呼ばれる。`noteOn` メソッドの内部で振幅変調をかける。`Line` クラスの変数 `ampEnv` から `activate` メソッドを利用し、音の長さの情報と開始時の振幅の大きさと終了時の振幅の大きさを引数として渡す。この時の開始時の振幅の大きさにコンストラクタ内で受け取った最大振幅を格納したフィールド（変数）`this.maxAmp` を指定することで、振幅の大きさを一音ごとに変更することができる。

ソースコード 4 Instrument インタフェースの実装

```

6  Waveform currentWaveform; // 音色格納用変数
7
8  // 音色を変更するためにInstrument インタフェースを実装する
9  class HackInstrument implements Instrument
10 {
11     Oscil wave;
12     Line ampEnv;
13     float maxAmp;
14
15     HackInstrument( float frequency, float maxAmp, Waveform wf )
16     { // Oscilを使って音信号を作成（周波数，振幅，音色）
17         wave = new Oscil( frequency, 0, wf );
18         // 引数で渡された最大振幅をクラスの変数に代入
19         this.maxAmp = maxAmp;
20         // 振幅変調を与える（初期値は1から0への減衰）
21         ampEnv = new Line( );
22         // 作成した音信号を振幅変調の出力に送る
23         ampEnv.patch( wave.amplitude );
24     }
25
26     // コールバック関数：再生開始
27     void noteOn( float duration )
28     { // 振幅変調の開始（長さ，開始時の振幅，終了時の振幅）
29         ampEnv.activate(duration, this.maxAmp, 0);
30         // 音の再生
31         wave.patch( out );
32     }
33
34     // コールバック関数：再生停止
35     void noteOff()
36     { // 再生の停止
37         wave.unpatch( out );
38     }
39 }

```

2.2 作成した instrument クラスを利用する

作成した `HackInstrument` クラスを利用して、`AudioOutput` に音符を追加するプログラムをソースコード 5 に示す。`HackInstrument` クラスを利用して音色を変更し、`AudioOutput` に音符を追加するには、`out.playNote` メソッドの第 3 引数に `HackInstrument` クラスのインスタンスを渡せばよい。コンストラクタの引数として周波数、最大振幅、音色の情報を渡している（60–61 行目）。この時、第 1 引数には音階の情報ではなく、音階の周波数を指定する必要があるため、`Frequency` クラスの `ofPith` メソッドと `asHz` メソッドを使用して、音階の情報（“A4”）を周波数（440 Hz）に変更している。

ソースコード 5 Instrument インタフェースを利用した音の追加

```

58 // 音を追加（開始時刻，音の長さ，Instrumentのインスタンス）

```

```

59 out.playNote(0.0f, 5.0f,
60     new HackInstrument(Frequency.ofPitch( "A4" ).asHz(),
61         0.5f, currentWaveform));

```

ソースコード 5 で `currentWaveform` として指定する音色の情報を、キー操作によって切り替えるプログラムをソースコード 6 に示す。キーと音色の対応は以下の通りである。

- 1: 正弦波
- 2: 三角波
- 3: のこぎり波
- 4: 矩形波
- 5: 倍音の振幅値を指定した自作の音色

自作の音色で指定する各倍音の取りうる値は 0.0–1.0 である。

ソースコード 6 キー操作で音色を変更

```

78 void keyPressed() {
79     switch (key)
80     {
81     case '1':
82         // 正弦波
83         currentWaveform = Waves.SINE;
84         break;
85     case '2':
86         // 三角波
87         currentWaveform = Waves.TRIANGLE;
88         break;
89     case '3':
90         // のこぎり波
91         currentWaveform = Waves.SAW;
92         break;
93     case '4':
94         // 矩形波
95         currentWaveform = Waves.SQUARE;
96         break;
97     case '5':
98         // 倍音の振幅値を指定した自作の音色
99         currentWaveform = WavetableGenerator.gen10(
100             4096, // サンプルサイズ (2の倍数で)
101             new float[] { 1.0f, 0.45f, 0.20f, 0.10f, 0.05f } // 各倍音の振幅値
102         );
103         break;
104     case 'p':
105         // 作成した信号を出力
106         playSong();
107         break;
108     default:
109         break;
110     }
111 }

```

3 メロディを生成する

メロディを生成する方法について説明する。このセクションに記載したソースコードを利用したプログラムの全文は、付録 B のソースコード 10 に記載した。異なる 3 つの音の高さ (C4(ド), D4(レ), E4(ミ)) の音信号を生成し、再生するためには、ソースコード 7 に示すように、音の高さ (melody)、音の長さ (duration)、音の開始位置 (startTime) を配列によって保持しておくが良い。

ソースコード 7 音の高さ、長さ、開始位置の情報を格納する配列

```

8 // 各音の高さ
9 String [] melody = {
10     "C4", "D4", "E4"
11 };
12
13 // 各音の長さ（拍）
14 float [] duration = {
15     0.9f, 0.9f, 0.9f
16 };
17
18 // 各音の開始位置
19 float [] startTime = {
20     0.0f, 1.0f, 2.0f
21 };

```

ソースコード 7 のように音の情報を格納した配列を使用し、繰り返し処理を用いてメロディを生成する（ソースコード 8）。

ソースコード 8 配列と繰り返し処理を使った音の追加

```

73 // 繰り返し処理を使って異なる音を追加
74 for (int i = 0; i < melody.length; i++) {
75     out.playNote(startTime[i], duration[i],
76         new HackInstrument(Frequency.ofPitch( melody[i] ).asHz(),
77             0.5f, currentWaveform));
78 }

```

付録 A 振幅と音色を変更するプログラム

ソースコード 9 振幅と音色を変更する

```
1  import ddf.minim.*;
2  import ddf.minim.ugens.*;
3
4  Minim minim;
5  AudioOutput out;
6  Waveform currentWaveform; // 音色格納用変数
7
8  // 音色を変更するためにInstrument インタフェースを実装する
9  class HackInstrument implements Instrument
10 {
11     Oscil wave;
12     Line ampEnv;
13     float maxAmp;
14
15     HackInstrument( float frequency, float maxAmp, Waveform wf )
16     { // Oscilを使って音信号を作成（周波数，振幅，音色）
17         wave = new Oscil( frequency, 0, wf );
18         // 引数で渡された最大振幅をクラスの変数に代入
19         this.maxAmp = maxAmp;
20         // 振幅変調を与える（初期値は1から0への減衰）
21         ampEnv = new Line( );
22         // 作成した音信号を振幅変調の出力に送る
23         ampEnv.patch( wave.amplitude );
24     }
25
26     // コールバック関数：再生開始
27     void noteOn( float duration )
28     { // 振幅変調の開始（長さ，開始時の振幅，終了時の振幅）
29         ampEnv.activate(duration, this.maxAmp, 0);
30         // 音の再生
31         wave.patch( out );
32     }
33
34     // コールバック関数：再生停止
35     void noteOff()
36     { // 再生の停止
37         wave.unpatch( out );
38     }
39 }
40
41 void setup()
42 {
43     size(512, 200);
44     // minimのインスタンスを用意
45     minim = new Minim(this);
46     // minimのgetLineOutメソッドを呼び出し，AudioOutputオブジェクトを受け取る
47     out = minim.getLineOut();
48     // テンポの設定，BPM=120
49     out.setTempo( 120 );
50     // 音色の初期値（正弦波）
51     currentWaveform = Waves.SINE;
52 }
53
54 void playSong() {
55     // 再生を停止
56     out.pauseNotes();
57     // 音を追加（開始時刻，音の長さ，Instrumentのインスタンス）
```

```

58 out.playNote(0.0f, 5.0f,
59     new HackInstrument(Frequency.ofPitch( "A4" ).asHz(),
60         0.5f, currentWaveform));
61 // 再生
62 out.resumeNotes();
63 }
64
65 void draw()
66 {
67     background(0);
68     stroke(255);
69
70     // 左チャンネルと右チャンネルに入っている波形を描画
71     for (int i = 0; i < out.bufferSize() - 1; i++)
72     {
73         line( i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50 );
74         line( i, 150 + out.right.get(i)*50, i+1, 150 + out.right.get(i+1)*50 );
75     }
76 }
77
78 void keyPressed() {
79     switch (key)
80     {
81     case '1':
82         // 正弦波
83         currentWaveform = Waves.SINE;
84         break;
85     case '2':
86         // 三角波
87         currentWaveform = Waves.TRIANGLE;
88         break;
89     case '3':
90         // のこぎり波
91         currentWaveform = Waves.SAW;
92         break;
93     case '4':
94         // 矩形波
95         currentWaveform = Waves.SQUARE;
96         break;
97     case '5':
98         // 倍音の振幅値を指定した自作の音色
99         currentWaveform = WavetableGenerator.gen10(
100             4096, // サンプルサイズ (2の倍数で)
101             new float[] { 1.0f, 0.45f, 0.20f, 0.10f, 0.05f } // 各倍音の振幅値
102         );
103         break;
104     case 'p':
105         // 作成した信号を出力
106         playSong();
107         break;
108     default:
109         break;
110     }
111 }

```

付録 B メロディを生成するプログラム

ソースコード 10 メロディの生成

```

1 import ddf.minim.*;
2 import ddf.minim.ugens.*;

```

```

3
4 Minim minim;
5 AudioOutput out;
6 Waveform currentWaveform; // 音色格納用変数
7
8 // 各音の高さ
9 String [] melody = {
10     "C4", "D4", "E4"
11 };
12
13 // 各音の長さ (拍)
14 float [] duration = {
15     0.9f, 0.9f, 0.9f
16 };
17
18 // 各音の開始位置
19 float [] startTime = {
20     0.0f, 1.0f, 2.0f
21 };
22
23 // 音色を変更するためにInstrument インタフェースを実装する
24 class HackInstrument implements Instrument
25 {
26     Oscil wave;
27     Line ampEnv;
28     float maxAmp;
29
30     HackInstrument( float frequency, float maxAmp, Waveform wf )
31     { // Oscilを使って音信号を作成 (周波数, 振幅, 音色)
32         wave = new Oscil( frequency, 0, wf );
33         // 引数で渡された最大振幅をクラスの変数に代入
34         this.maxAmp = maxAmp;
35         // 振幅変調を与える (初期値は1から0への減衰)
36         ampEnv = new Line( );
37         // 作成した音信号を振幅変調の出力に送る
38         ampEnv.patch( wave.amplitude );
39     }
40
41     // コールバック関数: 再生開始
42     void noteOn( float duration )
43     { // 振幅変調の開始 (長さ, 開始時の振幅, 終了時の振幅)
44         ampEnv.activate(duration, this.maxAmp, 0);
45         // 音の再生
46         wave.patch( out );
47     }
48
49     // コールバック関数: 再生停止
50     void noteOff()
51     { // 再生の停止
52         wave.unpatch( out );
53     }
54 }
55
56 void setup()
57 {
58     size(512, 200);
59     minim = new Minim(this);
60     // minimのインスタンスを用意
61     minim = new Minim(this);
62     // minimのgetLineOutメソッドを呼び出し, AudioOutputオブジェクトを受け取る
63     out = minim.getLineOut();
64     // テンポの設定, BPM=120
65     out.setTempo( 120 );

```



```

66 // 音色の初期値 (正弦波)
67 currentWaveform = Waves.SINE;
68 }
69
70 void playSong() {
71 // 再生を停止
72 out.pauseNotes();
73 // 繰り返し処理を使って異なる音を追加
74 for (int i = 0; i < melody.length; i++) {
75     out.playNote(startTime[i], duration[i],
76         new HackInstrument(Frequency.ofPitch( melody[i] ).asHz(),
77             0.5f, currentWaveform));
78 }
79 // 再生
80 out.resumeNotes();
81 }
82
83 void draw()
84 {
85     background(0);
86     stroke(255);
87
88 // 左チャンネルと右チャンネルに入っている波形を描画
89 for(int i = 0; i < out.bufferSize() - 1; i++)
90 {
91     line( i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50 );
92     line( i, 150 + out.right.get(i)*50, i+1, 150 + out.right.get(i+1)*50 );
93 }
94 }
95
96 void keyPressed() {
97     switch (key)
98     {
99         case '1':
100             // 正弦波
101             currentWaveform = Waves.SINE;
102             break;
103         case '2':
104             // 三角波
105             currentWaveform = Waves.TRIANGLE;
106             break;
107         case '3':
108             // のこぎり波
109             currentWaveform = Waves.SAW;
110             break;
111         case '4':
112             // 矩形波
113             currentWaveform = Waves.SQUARE;
114             break;
115         case '5':
116             // 16個の奇数倍音からなる音。振幅値はランダム。
117             currentWaveform = Waves.randomNOddHarms(16);
118             break;
119         case '6':
120             // 倍音の振幅値を指定した自作の音色
121             currentWaveform = WavetableGenerator.gen10(
122                 4096, // サンプルサイズ (2の倍数で)
123                 new float[] { 1.0f, 0.45f, 0.20f, 0.10f, 0.05f } // 各倍音の振幅値
124                     (0.0~1.0)
125             );
126             break;
127         case 'p':
128             // 作成した信号を出力

```

```
128     playSong();  
129     break;  
130     default: break;  
131 }  
132 }
```